Machine Learning II
TP Réseaux de neurones artificiels
Etude de cas : Analyse de la rétention des clients bancaires

Dans cette activité, vous allez utiliser un réseau de neurones artificiel (ANN) pour prédire si un client d'une banque est susceptible de quitter la banque ou non.

Jeu de données

Le jeu de données avec lequel vous allez travailler est Bank_customers.csv qui est fourni avec cet énoncé. Vous devriez vérifier ce que chaque colonne représente et déterminer si c'est une information pertinente à utiliser comme une caractéristique ou non.

Consignes

1. Ouvrez le notebook Python ANN_VStd.ipynb et complétez les parties manquantes du code.

   NB : Dans ce notebook Python, vous trouverez les instructions dans des cellules de contenu textuel ainsi que des cellules de code exécutable dans lesquelles vous ajouterez votre code.
   N'oubliez pas de vérifier les résultats obtenus à chaque étape.
   Notez que ces résultats sont déjà affichés dans le notebook pour vous aider à mieux comprendre ce que vous devez implémenter.
   Attention : Il est important d'ajouter des commentaires internes pour expliquer votre code et interpréter les résultats obtenus. Cela sera également pris en compte lors de l'évaluation.

   Suivez les instructions données dans le notebook pour résoudre ce problème :

2. Chargez le jeu de données.

3. Inspectez et nettoyez les données si nécessaire. Vérifiez ce que chaque colonne dans le jeu de données représente et déterminez si c'est une information pertinente à utiliser comme une caractéristique ou non. Supprimez donc les caractéristiques non désirées.

4. Encodez les caractéristiques catégorielles en valeurs numériques.

5. Divisez vos données en jeux de données d'entraînement et de test.

6. Utilisez StandardScaler pour mettre à l'échelle vos données.

7. Installez Keras au-dessus de TensorFlow.

8. Utilisez Sequential de keras.models pour créer un modèle ANN sous forme d'une séquence de couches.

9. Ajouter la couche d'entrée et la première couche cachée en utilisant la méthode add().

```
classifier.add(Dense(units=6, kernel_initializer='uniform', activation='relu', input_dim=11))
```

Attention : si le nombre de variables retenues après le prétraitement dans votre jeu de données est différent de 11, veuillez modifier le nombre dans l'argument input_dim en conséquence.

Utilisez Dense() pour créer une couche Dense dans votre réseau de neurones. connecté habituelle. C'est le type de couche le plus courant et le plus fréquemment utilisé.
La méthode Dense prend les arguments suivants :

- `units`: dimension de l'espace de sortie.
- `activation`: fonction d'activation à utiliser. Dans cette activité, utilisez :
  - 'relu' : fonction d'activation rectifieuse pour les couches cachées.
  - 'sigmoid' pour la couche de sortie, qui peut être utilisée lorsque la sortie est binaire.
- `kernel_initializer`: Initialisateur pour la matrice des poids du noyau.
- Attention : L'argument `input_dim`, qui représente la dimension de la couche d'entrée, est à passer seulement pour la première couche cachée.

10. Appliquez Dropout sur les unités d'entrée pour la régularisation avec une probabilité de 0.1 pour éviter le surapprentissage.

11. Ajoutez une deuxième couche cachée composée de 6 unités. Appliquez également Dropout.

12. Ajoutez la couche de sortie avec une unité binaire. N'oubliez pas que cette fois-ci, vous devez utiliser la fonction d'activation sigmoid.

13. Avant de former le modèle ANN, vous devez configurer le processus d'apprentissage, ce qui se fait en utilisant la méthode compile.

```
classifier.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

compile() reçoit trois arguments : un optimiseur (utilisez 'adam'), une fonction de perte (utilisez 'binary_crossentropy'), et une liste de métriques (utilisez 'accuracy'). Cela signifie que votre processus de validation sera effectué en utilisant l'optimiseur Adam qui utilise la descente de gradient stochastique pour optimiser les hyperparamètres du réseau, principalement les poids des caractéristiques. L'erreur est calculée à l'aide de la fonction de perte 'binary_crossentropy', et la métrique d'évaluation utilisée est le score 'accuracy'.

14. Entraînez votre modèle et appliquez une validation croisée avec une taille de lot de 10. Répétez 50 ou 100 fois (epoch=50 ou 100).

```
classifier.fit(X_train, y_train, batch_size=10, epochs=50)
```

15. Faites des prédictions.

16. Évaluez le modèle en utilisant la matrice de confusion et le rapport de classification.

17. Prédisez si le client avec les informations suivantes quittera ou non la banque :

- Géographie : France
- Score de crédit : 600
- Genre : Homme
- Âge : 40
- Durée de possession du compte : 3
- Solde : 60 000
- Nombre de produits : 2
- Possède une carte de crédit : Oui
- Est un membre actif : Oui
- Salaire estimé : 50 000

18. Vous pouvez utiliser les modèles séquentiels Keras dans le cadre de votre flux de travail Scikit-Learn via les wrappers trouvés dans la bibliothèque Keras. Suivez les consignes données dans le notebook Python pour comprendre comment utiliser KerasClassifier à partir de keras.wrappers.scikit_learn.

19. Appliquez la validation croisée k-fold pour l'évaluation.

Le code de ces dernières parties de l'activité vous est fourni pour vous aider à comprendre comment construire un modèle d'apprentissage en profondeur pour la première fois.

NB : N'oubliez pas d'ajouter des commentaires internes pour l'interprétation et l'explication à chaque étape. Cela est obligatoire et sera noté.